

TASK 01 — Spring Security Fundamentals & Authentication Setup

Step 1 — Study security fundamentals

Authentication

Authentication means:

"Who are you"

The system verifies the user's identity.

Example:

Username: madhava
Password: 12345

If the username and password are correct:

Authentication successful

Common authentication methods:

- Username + Password
- JWT Token
- OAuth2
- API Key

For our Spring Boot project, we can start with **username/password authentication** and later implement **JWT**.

Authorization

Authorization means:

"What are you allowed to do"

For example:

User → Can view products
Admin → Can view, create, update and delete products

So:

Authentication → Who are you?

Authorization → What can you access?

Identity

Identity represents the user who has authenticated.

Example:

Username: john
Role: USER

Spring Security can keep the authenticated user's information in the **SecurityContext**.

Roles

A role defines a user's category.

For example:

USER
ADMIN
MANAGER

Example:

john → USER
madhava → ADMIN

We can then restrict APIs based on roles.

Example:

```
.requestMatchers("/api/admin/**").hasRole("ADMIN")
```

This means only users with the ADMIN role can access `/api/admin/**`.

Permissions

Permissions provide more specific access control.

PRODUCT_READ
PRODUCT_CREATE
PRODUCT_UPDATE
PRODUCT_DELETE

An admin could have:

PRODUCT_READ
PRODUCT_CREATE

PRODUCT_UPDATE
PRODUCT_DELETE

while a normal user might have:

PRODUCT_READ

Role = broader category.

Permission = specific action.

Access Control

Access control determines:

Public vs Protected APIs

Some APIs should be publicly accessible.

Public API

GET /api/products

No login required.

Protected API

POST /api/products

Login required.

Admin-only API

DELETE /api/products/1

Admin role required.

So our application can have:

Public



Authenticated



Authorized



Controller

Step 2 — Understand Spring Security Architecture

Client

The client can be:

- Postman
- Browser
- Frontend application
- Mobile application

Example, Postman sends:

GET /api/products

Security Filter

Before the request reaches your controller, **Spring Security checks the request.**

Conceptually:

Request



Spring Security Filter



Check authentication



Check authorization



Controller

This is important because security checks happen **before your controller method executes.**

Authentication

Spring Security checks whether the request has valid credentials.

For example:

Username + Password

or later:

Authorization: Bearer <JWT_TOKEN>

If authentication fails:

401 Unauthorized

Authorization

After authentication, Spring Security checks permissions/roles.

Example:

User = USER

Required = ADMIN

The user is authenticated, but doesn't have permission.

Result:

403 Forbidden

Remember:

401 → Authentication problem

403 → Authorization problem

Controller

Only after security checks succeed does the request reach the controller.

For example:

```
@GetMapping("/products")
public List<Product> getProducts() {
    return productService.getProducts();
}
```

For your **Spring Boot Product API**, we can build security step-by-step.

Phase 1 — Add Spring Security

Add Spring Security dependency.

For Maven:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

This adds Spring Security to the application.

Phase 2 — Create Public and Protected APIs

For example:

GET /api/products

Make this public.

POST /api/products

Make this protected.

PUT /api/products/{id}

Make this admin-only.

DELETE /api/products/{id}

Make this admin-only.

Phase 3 — Configure Security

Create:

SecurityConfig.java

Example basic configuration:

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Bean

 SecurityFilterChain securityFilterChain(HttpSecurity http)

 throws Exception {

 http

 .csrf(csrf-> csrf.disable())

 .authorizeHttpRequests(auth-> auth

 .requestMatchers(HttpMethod.GET, "/api/products").permitAll()

 .requestMatchers(HttpMethod.POST, "/api/products").hasRole("ADMIN")

 .requestMatchers(HttpMethod.PUT, "/api/products/**").hasRole("ADMIN")

 .requestMatchers(HttpMethod.DELETE, "/api/products/**").hasRole("ADMIN")

```
        .anyRequest().authenticated()
    )
    .httpBasic(Customizer.withDefaults());

    return http.build();
}
}
```

This tells Spring Security:

GET products



PUBLIC

POST products



ADMIN

PUT products



ADMIN

DELETE products



ADMIN

Everything else



LOGIN REQUIRED

Phase 4 — Create Users

For learning purposes, we can create users in memory.

@Bean

```
UserDetailsService users(PasswordEncoder encoder) {  
    UserDetails user = User.builder()  
        .username("user")  
        .password(encoder.encode("user123"))  
        .roles("USER")  
        .build();  
  
    UserDetails admin = User.builder()  
        .username("admin")  
        .password(encoder.encode("admin123"))  
        .roles("ADMIN")  
        .build();  
  
    return new InMemoryUserDetailsManager(user, admin);  
}
```

Then:

user / user123

has:

ROLE_USER

and:

admin / admin123

has:

ROLE_ADMIN

Phase 5 — Password Encoding

Never store passwords like:

admin123

directly in a real application.

Use a password encoder:

@Bean

```
PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

Then:

```
.password(encoder.encode("admin123"))
```

Spring Security stores a BCrypt hash rather than the plain password.

Phase 6 — Test Using Postman

You can test each security scenario.

Test 1 — Public API

GET <http://localhost:8083/api/products>

Expected:

200 OK

No username/password required.

Test 2 — Protected API without login

POST <http://localhost:8083/api/products>

No credentials.

Expected:

401 Unauthorized

Test 3 — Normal USER trying admin API

Login as:

user

user123

Try:

DELETE [/api/products/1](http://localhost:8083/api/products/1)

Expected:

403 Forbidden

Because the user is authenticated but doesn't have ADMIN permission.

Test 4 — ADMIN

Login as:

admin

admin123

Try:

DELETE /api/products/1

Expected:

200 OK

or:

204 No Content

depending on your controller.

Step 3 — Add Spring Security

We add Spring Security to the existing Spring Boot project.

Before security:

Client → Controller → Service → Database

After adding Spring Security:

Client

↓

Spring Security

↓

Authentication / Authorization

↓

Controller

↓

Service

↓

Database

Add dependency

If we are using **Maven**, open:

pom.xml

Add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Your dependencies might look like:

```
<dependencies>
  <!-- Spring Web-->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- Spring Data JPA-->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <!-- Spring Security-->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
</dependencies>
```

Then reload Maven.

In IntelliJ:

Maven → Reload Project

or in Eclipse:

Right Click Project

→ Maven

→ Update Project

Step 4 — Run the Application

Now start your Spring Boot application.

Example:

ProductServiceApplication.java

@SpringBootApplication

```
public class ProductServiceApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(ProductServiceApplication.class, args);  
    }  
}
```

If your application runs on port 8083:

<http://localhost:8083>

Before adding Spring Security:

GET /api/v1/products

↓

200 OK

After adding Spring Security, Spring Security protects endpoints **by default**.

You may see:

401 Unauthorized

when accessing:

GET /api/v1/products

without authentication.

Spring Security may also generate a default development password in the console when no custom authentication configuration is provided.

Step 5 — Configure SecurityFilterChain

Create a package:

config

Inside it create:

SecurityConfig.java

SecurityConfig.java

```
package com.example.product.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http
            .csrf(csrf-> csrf.disable())
            .authorizeHttpRequests(auth-> auth
                // Public API
                .requestMatchers(
                    HttpMethod.GET,
                    "/api/v1/products/public"
                ).permitAll()
                // Protected APIs
                .requestMatchers(
```

```

        HttpMethod.POST,
        "/api/v1/products"
    ).authenticated()
    .requestMatchers(
        HttpMethod.PUT,
        "/api/v1/products/{id}"
    ).authenticated()
    .requestMatchers(
        HttpMethod.DELETE,
        "/api/v1/products/{id}"
    ).authenticated()
    // Everything else requires authentication
    .anyRequest().authenticated()
)
.httpBasic(Customizer.withDefaults());
return http.build();
}
}

```

Understand this code

`.authorizeHttpRequests(auth-> auth`

This tells Spring Security that we are going to define **access rules**.

Public API

```

.requestMatchers(
    HttpMethod.GET,
    "/api/v1/products/public"
).permitAll()

```

Means:

GET /api/v1/products/public

can be accessed by anyone.

No username/password required.

Client



GET /api/v1/products/public



Spring Security



permitAll()



Controller

Step 6 — Create Public Endpoint

Now create the endpoint in your ProductController.

Example:

```
@RestController
```

```
@RequestMapping("/api/v1/products")
```

```
public class ProductController {
```

```
    @GetMapping("/public")
```

```
    public String publicProducts() {
```

```
        return "This is a public product API";
```

```
    }
```

```
}
```

Now your endpoint is:

GET /api/v1/products/public

Test in Postman

Open Postman.

Select:

GET

Enter:

http://localhost:8083/api/v1/products/public

Click:

Send

Expected:

200 OK

Response:

This is a public product API

No authentication is required.

Step 7 — Protect Private Endpoints

Product Controller

@RestController

@RequestMapping("/api/v1/products")

public class ProductController {

 // PUBLIC

 @GetMapping("/public")

 public String publicProducts() {

 return "Public products";

 }

 // PRIVATE

 @PostMapping

 public Product createProduct(@RequestBody Product product) {

 return productService.createProduct(product);

 }

 // PRIVATE

 @PutMapping("/{id}")

 public Product updateProduct(

 @PathVariable Long id,

```

        @RequestBody Product product) {
            return productService.updateProduct(id, product);
        }
// PRIVATE
    @DeleteMapping("/{id}")
    public String deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return "Product deleted successfully";
    }
}

```

Now test everything in Postman

Test 1 — Public API

GET <http://localhost:8083/api/v1/products/public>

Authorization:

No Auth

Expected:

200 OK

Test 2 — POST without authentication

POST <http://localhost:8083/api/v1/products>

Authorization:

No Auth

Expected:

401 Unauthorized

This proves the endpoint is protected.

Test 3 — POST with authentication

In Postman:

Authorization

→ Basic Auth

Username:

user

Password:

user123

Then:

POST <http://localhost:8083/api/v1/products>

Example JSON:

```
{  
  "name": "Laptop",  
  "price": 55000  
}
```

Expected:

200 OK

or:

201 Created

depending on your controller implementation.

Test 4 — PUT

Request:

PUT <http://localhost:8083/api/v1/products/1>

Authorization:

Basic Auth

Username:

user

Password:

user123

Expected:

200 OK

Test 5 — DELETE

Request:

DELETE http://localhost:8083/api/v1/products/1

Authorization:

Basic Auth

Username:

user

Password:

user123

Expected:

200 OK

or:

204 No Content

depending on your implementation.

Step 8 — Test Without Authentication

Start Spring Boot

Run your application.

Example:

ProductServiceApplication

If your application uses port 8083:

http://localhost:8083

Open Postman

Create a new request.

Select: POST

Enter: http://localhost:8083/api/v1/products

Go to: Authorization

Select: No Auth

Then click: Send

Expected result

You should get:

401 Unauthorized

This is the exact result mentioned in your screenshot.

we get 401

Because you are doing:

POST /api/v1/products

but you're not providing credentials.

Spring Security sees:

Authentication?

↓

NO

↓

401 Unauthorized

The request does **not** reach your controller.

That's an important point.

Postman

↓

Spring Security

↓

401

↓

Controller is NOT executed

Test PUT without authentication

In Postman: PUT

URL: <http://localhost:8083/api/v1/products/1>

Authorization: No Auth

Click **Send**.

Expected:

401 Unauthorized

Test DELETE without authentication

Select: DELETE

URL: <http://localhost:8083/api/v1/products/1>

Authorization: No Auth

Click **Send**.

Expected:

401 Unauthorized

Test the public endpoint

Now test: GET

<http://localhost:8083/api/v1/products/public>

Authorization: No Auth

Click **Send**.

Expected:

200 OK

Why:

Because we configured:

`.permitAll()`

Test with authentication

Now test the same POST API again.

Request:

POST

<http://localhost:8083/api/v1/products>

Go to:

Authorization

↓

Basic Auth

Enter:

Username: user

Password: user123

Then click **Send**.

Now Spring Security sees:

Username/password



Authentication successful



Controller



Service



Database

You should receive your controller's normal success response, such as:

201 Created

or:

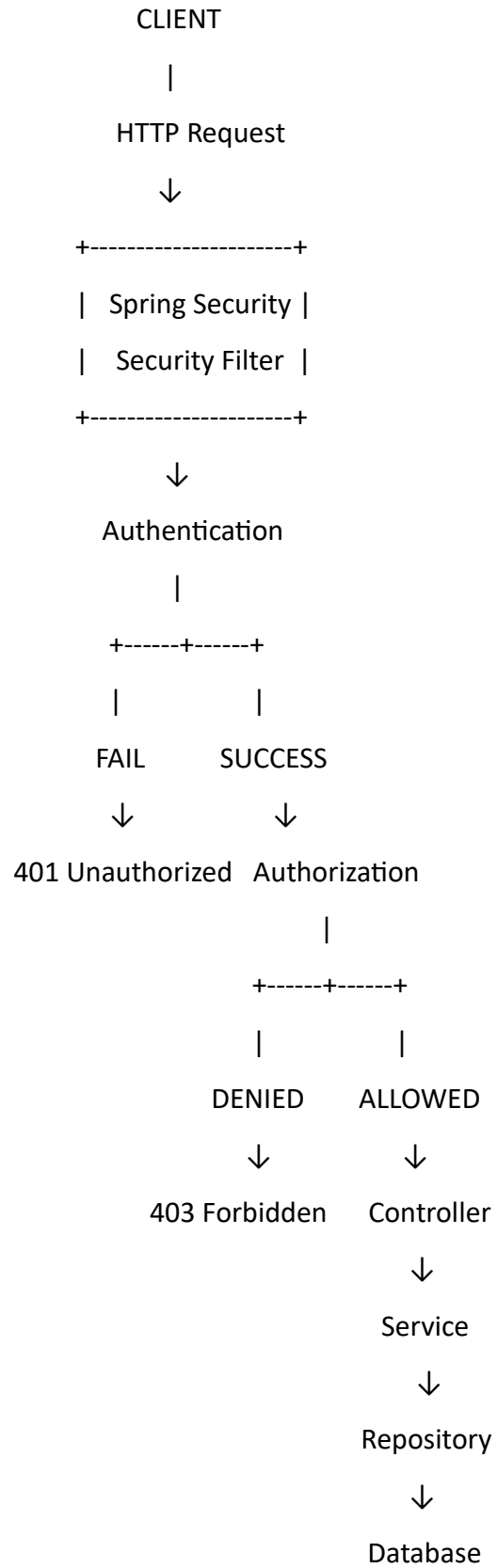
200 OK

depending on your implementation.

Step 9 — Document the Security Flow

Create a simple security architecture diagram.

Document it like this:



Public API Flow

For the public endpoint:

Client



GET /api/v1/products/public



Spring Security



permitAll()



Controller



Response

No login is required.

Private API Flow

For: POST /api/v1/products

the flow is:

Client



POST /api/v1/products



Spring Security



Authentication

└─ No credentials



| 401 Unauthorized

└─ Valid credentials



Controller



Service



Database